Project Report: Static Code Quality Analyzer for C

1. Introduction

This project is a custom **Static Code Quality Analyzer (SCA)** designed to detect bugs, code smells, and security vulnerabilities in C code without executing it. It operates entirely offline and provides detailed, CI/CD-friendly diagnostics and code grading.

2. Architecture Overview

The system is built using a modern **two-stage architecture** to combine the performance of C with the rapid analysis capabilities of Python: 1. **Frontend (`cparse`)**: Written in C, using **Flex** and **Bison**. It reads C source code, parses it according to our defined C grammar subset, and outputs an Abstract Syntax Tree (AST) in a strict JSON format. 2. **Backend Engine (`sca`)**: Written in Python. It ingests the JSON AST, maps it into Python objects, builds a **Control Flow Graph (CFG)**, executes advanced **Dataflow Analysis**, and runs 18 distinct semantic rules to grade the code.

---

3. The Frontend: Lexer and Parser (C, Flex, Bison)

3.1 Lexical Analysis (Flex)

The `lexer.l` file defines the tokens. It safely extracts keywords (`int`, `if`, `return`), identifiers, integers, floating-point literals, and operators. * **Comments & Directives**: The lexer safely ignores `//` and `/* */` comments to keep the AST pure. It also traps `#include` and `#define` directives in an exclusive state to capture them without breaking standard tokenization. * **Coordinate Tracking**: It strictly tracks `line` and `col` boundaries using `yylloc` so the Python backend knows exactly where to highlight errors.

3.2 Parsing (Bison)

The `parser.y` file contains the formal Context-Free Grammar. * **Conflict Resolution**: It resolves the classic **"Dangling-Else"** ambiguity by assigning explicit precedence weights (`%nonassoc LOWER_THAN_ELSE` vs `%nonassoc ELSE`). * **Memory Management**: To avoid memory fragmentation, the parser uses a custom **Arena Allocator**. It builds the AST in memory and then a JSON serializer recursively walks this tree, spitting out the final `stdout` payload.

---

4. The Backend: Semantic Engine (Python)

4.1 AST Mapping & The Visitor Pattern

The Python CLI intercepts the C JSON output and recursively maps it into Python `dataclasses` (e.g., `FunctionDef`, `If`, `Binary`). The rules engine heavily relies on the **Visitor Pattern**, allowing recursive algorithms to cleanly traverse deeply nested loops and blocks.

4.2 Control Flow Graph (CFG)

To analyze execution paths, the AST is converted into a **CFG**. * **Basic Blocks**: Straight-line sequences of statements without branches. * **Edges**: Directed connections representing control flow. * **Complex Routing**: The `CFGBuilder` accurately bridges blocks for complex structures like `while`, `do-while`, and `switch`. It uses internal stacks to correctly route `break` and `continue`

jumps to the correct loop headers or exit nodes.

4.3 Dataflow Analysis (Worklist Algorithm)

Advanced bugs (like dead stores and uninitialized variables) cannot be found by simply reading the code top-to-bottom. We implemented a **Worklist Solver** to execute Dataflow equations over the CFG. * **Liveness Analysis (Backward)**: Computes which variables are "live" (needed in the future). It walks backward through the CFG, updating `IN` and `OUT` sets. If a variable is written to, but it does not exist in the immediate `OUT` liveness set, it is flagged as a **Dead Store**. * **Uninitialized Variables (Forward)**: Computes variable states moving forward. If an identifier is read before it is explicitly assigned in the flow path, it is flagged.

---

5. Rules Engine Highlights

The analyzer evaluates the AST and CFG against 18 distinct rules divided into categories: 1. **AST Rules**: Catch structural issues like `deep-nesting` (> 4 levels), `function-too-long` (> 50 lines), and `too-many-params`. 2. **Symbol-Table Rules**: Flag variable shadowing and array index out-of-bounds (by evaluating `IntLiteral` accesses against static array declarations). 3. **CFG Rules**: Detects `unreachable-code` (blocks completely disconnected from the entry node) and `missing-return` (paths flowing to the exit block without hitting a return statement). 4. **Dataflow Rules**: Captures inefficient `dead-stores` and dangerous `uninitialized` accesses. 5. **Security Rules**: The `unsafe-function` rule bans functions prone to buffer overflows (like `gets` and bare `scanf("%s")`), recommending safer alternatives like `fgets`.

---

6. CLI Features and Polish

- **Inline Suppression**: To allow deliberate exceptions, a Python regex-parser scans the raw source code for `// sca-disable-next-line: <rule>`. Any flagged issue on that line is quietly pruned before the final output.
- **Quality Scoring**: The `sca score` command aggregates penalties based on issue severity (High severity deducts 10 points, minor issues deduct 1 point) and outputs a final grade out of 100 (A to F).

7. Conclusion

By decoupling the heavy, complex parsing logic into C (Flex/Bison) and the semantic graphing algorithms into Python, the project successfully implements enterprise-grade static analysis. It correctly evaluates structural aesthetics, control flow integrity, and dataflow liveness in a robust, automated pipeline.